

Entregable N°11:

Integración Hardware - Software - Manufactura digital

Autores:

Alessandro Nicolas Crosby Collantes

Pabel Mario Condori Pompilio

Arianna Fabiana Del Valle Fuentes Contreras

Paola Andrea Fernández García

Brandy Abigail Cordova Palomino

Sebastián Amadeus Espinoza Padilla

Asesores:

Miguel Rogger Hoyos Alvitez

Marco Mugaburu Celi

Shirley Pahuachon Nuñez

Curso:

Fundamentos de Biodiseño – Ciclo IV

Universidad Peruana Cayetano Heredia

Facultad de Ciencias e Ingeniería

Noviembre 2025

Verificación de Diseño - SOFTWARE

Tabla de Verificación de Diseño (Software)

Requisito Funcional / No Funcional	Descripción del Componente o Función a Verificar	Método de Verificación (Prueba / Revisión / Simulación)	Criterio de Aceptación	Resultado (Cumple / No cumple)	Observaciones / Recomendaciones
RF1 – Captar y procesar señales oculares	El sistema debe reconocer la dirección de la mirada y traducirla en comandos.	Prueba funcional en entorno controlado con cámara Android.	Reconocimiento $\geq 90\%$ en entorno iluminado.	Cumple parcialmente	Precisión baja en baja luz; se recomienda calibración adaptativa.
RF2 – Controlar comandos de movimiento	La app debe interpretar el movimiento ocular para enviar comandos	Simulación con 4 direcciones (arriba, abajo, izq, der)	Tasa de error $< 10\%$.	Cumple	Fluidez adecuada, sin retrasos perceptibles.
RF3 – Transmisión Bluetooth	Comunicación entre app Android y módulo ESP32	Prueba de conexión continua por 60 min.	Pérdida de conexión $< 1\%$	Cumple	Comunicación estable durante test prolongado.
RF4 – Interpretación de comandos (ESP32)	Software debe enviar datos correctos a microcontrolador.	Revisión de código y trazado serial.	Señal reconocida en $> 95\%$ de casos	Cumple	Comunicación estable, latencia promedio < 100 ms.
RF5 – Interfaz de usuario	Interfaz visual clara y accesible	Prueba de usabilidad con usuario simulado	Usuario identifica comandos sin asistencia.	Cumple	Diseño intuitivo, buen contraste visual.
RF6 – Seguridad en fallos	Parada automática en error de conexión.	Simulación de desconexión intencional	Detención inmediata del sistema.	Cumple	Sistema detiene movimiento al perder conexión.

RNF1 – Usabilidad	Curva de aprendizaje ≤15 min.	Evaluación con usuario simulado	100% comprende funciones básicas en <15 min.	Cumple	Interfaz intuitiva, requiere capacitación mínima.
RNF2 – Rendimiento	Procesamiento fluido sin retrasos notables.	Cronometría de respuesta visual–acción.	Latencia ≤0.2 s.	Cumple	Respuesta inmediata a estímulos visuales.
RNF3 – Fiabilidad	Prueba de operación continua 60 min.	Prueba de estrés.	Sin reinicios ni cuelgues.	Cumple parcialmente	Leve ralentización tras 45 min; optimizar gestión de memoria
RNF4 – Seguridad del sistema	Evita activaciones involuntarias	Pruebas de error ocular y parpadeo.	Activaciones <5%.	Cumple	Confirmación por doble fijación evita falsos positivos.

Tabla de Verificación del Prototipo con Funcionalidades Mixtas (Software / Hardware)

Subsistema	Elemento Verificado	Tipo de Componente	Criterio de Verificación	Resultado Esperado	Resultado Obtenido	Cumple / No cumple	Observaciones
Eye-Tracing – App	Procesamiento y transmisión de señales	Software + Sensor	Datos reconocidos y enviados correctamente vía Bluetooth	Señales transmitidas sin pérdida	Transmisión estable	Cumple	Comunicación óptima entre módulos.
App Android – ESP32	Comunicación bidireccional	Software + Hardware	Sincronización entre interfaz y microcontrolador	Sin retrasos >0.2 s	Latencia 0.1 s	Cumple	Sistema fluido y estable.
ESP32 – Servomotores	Ejecución de comandos	Software embebido	Comando recibido = movimiento correcto	Movimiento preciso del joystick	Movimiento suave y exacto	Cumple	Se mantiene control direccional correcto.

Botón de emergencia	Parada inmediata	Hardware + Software	Detención automática ante error	Sistema detiene acción	Cumple	Cumple	Función de seguridad validada.
Sensores ultrasónicos	Detección de obstáculos	Hardware + Software	Emisión de alerta y corrección de trayectoria	Corrección automática al detectar objeto	Parcial	No cumple totalmente	Falta respuesta en ángulo ciego lateral.

Evaluación de Requerimientos No Cumplidos o a Replantear

Requisito	Situación Actual	Motivo del Incumplimiento / Riesgo	Acción Correctiva o Replanteamiento
RF1 – Precisión del eye-tracking	Cumple parcialmente	Variabilidad por condiciones de iluminación	Implementar calibración dinámica o sensor infrarrojo.
RNF3 – Fiabilidad (uso prolongado)	Cumple parcialmente	Gestión ineficiente de memoria en app	Optimizar código y liberar buffers tras cada sesión.
Sensor ultrasónico lateral	No cumple completamente	Ángulo muerto no cubierto por el sensor	Añadir sensor adicional o algoritmo de fusión de datos.

Verificación de Diseño - Hardware

Tabla de Verificación de Diseño (Hardware)

Componente / Subsistema	Método de verificación	Criterio de aceptación	Resultado	Observaciones / Recomendaciones
Carcasa + ESP32 + batería + drivers + cableado	Medición con balanza	≤ 0.80 kg	Cumple	Aunque con las bases que se tenía cumple, se realizarán pruebas con todo completo
Batería + consumo total	Prueba en uso continuo (movimientos/ BT)	≥ 4 h	No cumple totalmente	Realizar unas pruebas más para tener bien en claro

E-STOP + lógica de corte	Prueba física: pulsación durante movimiento	Detención inmediata	Cumple	Se detiene si la persona no esta mirando ademas de desactivar el mecanismo en cualquier momento con el switch
Power / Reset accesibles	Inspección + prueba	Accesibles sin herramientas	Cumple	Reubicar si hay interferencias con el apoyabrazos, aun se realizara prueba.
Carcasa y anclajes	Inspección + plantilla	No invade ruedas/apoyabrazos; no juego >2 mm	Cumple	En teoría cumple pero se realizará prueba.
Carcasa + soportes	Prueba de vibración/golpe leve	Sin fisuras; tornillos sin aflojar	Cumple	En teoría cumple pero se realizará prueba.
Servomotores + acople a joystick	Prueba funcional con cargas	Par $\geq$ requerido; seguimiento suave	Cumple	En teoría cumple pero se realizará prueba.
LEDs/buzzer	Prueba de patrones	Estados distinguibles (conectado, error, stop)	Cumple	En la aplicación podrá observarse el estado
Drivers/servos /carcasa	Medición con termómetro IR	Carcasa <45 °C; driver <70 °C	Cumple	No se usará ventilación pero las siguientes pruebas de uso lo determinaran mejor

Vídeo del funcionamiento piloto de EyePilot:

<https://youtube.com/shorts/qDQIfGdBrkY?feature=share>

Foto del prototipo:



Código del Arduino IDE:

```
1  #include <ESP32Servo.h>
2  #include "BluetoothSerial.h"
3  #include "esp_system.h"
4  #include "esp_mac.h"
5
6  // Comprobar si el Bluetooth está activado en la configuración
7  #if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
8  #error Bluetooth is not enabled! Please run 'make menuconfig' to enable it
9  #endif
10
11 // Objeto para la comunicación Bluetooth
12 BluetoothSerial SerialBT;
13
14 // Pines
15 const int pinServo1 = 18;
16 const int pinServo2 = 19;
17
18 // --- Objetos Servo ---
19 Servo servo1;
20 Servo servo2;
21
22 // --- Posiciones definidas para los servos ---
23 const int SERVO_X_MIN = 0;
24 const int SERVO_X_CENTRO = 85;
25 const int SERVO_X_MAX = 174;
26
27 const int SERVO_Y_MIN = 171;
28 const int SERVO_Y_CENTRO = 85;
29 const int SERVO_Y_MAX = 0;
30
31
32 void setup() {
33     Serial.begin(115200);
34
35     // IMPRIMIR DIRECCIÓN MAC DE BLUETOOTH
36     Serial.println("--- Obteniendo Dirección MAC de Bluetooth ---");
37     uint8_t mac[6];
38     esp_read_mac(mac, ESP_MAC_BT); // Ahora esto funcionará
39
40     Serial.print("Mi dirección MAC de Bluetooth es: ");
41     for (int i = 0; i < 6; i++) {
42         if (mac[i] < 0x10) {
43             Serial.print("0");
44         }
45         Serial.print(mac[i], HEX);
46         if (i < 5) {
47             Serial.print(":");
48         }
49     }
50     Serial.println();
51     Serial.println("-----");
52     // FIN DE IMPRIMIR MAC
53
54     SerialBT.begin("EyePilot_ESP32");
55     Serial.println("Bluetooth iniciado. Listo para conectar.");
56
57     servo1.attach(pinServo1, 500, 2500);
58     servo2.attach(pinServo2, 500, 2500);
59
60     servo1.write(SERVO_X_CENTRO);
61     servo2.write(SERVO_Y_CENTRO);
62
63     Serial.println("Servos centrados. Esperando comandos Bluetooth...");
64 }
65
```

```

66 void loop() {
67   if (SerialBT.available()) {
68     String cmd = SerialBT.readStringUntil('\n');
69     cmd.trim();
70
71     Serial.print("Comando recibido: [");
72     Serial.print(cmd);
73     Serial.println("]");
74
75     if (cmd == "STOP") {
76       servo1.write(SERVO_X_CENTRO);
77       servo2.write(SERVO_Y_CENTRO);
78     }
79     else if (cmd == "UP") {
80       servo2.write(SERVO_Y_MAX);
81       servo1.write(SERVO_X_CENTRO);
82     }
83     else if (cmd == "DOWN") {
84       servo2.write(SERVO_Y_MIN);
85       servo1.write(SERVO_X_CENTRO);
86     }
87     else if (cmd == "LEFT") {
88       servo1.write(SERVO_X_MIN);
89       servo2.write(SERVO_Y_CENTRO);
90     }
91     else if (cmd == "RIGHT") {
92       servo1.write(SERVO_X_MAX);
93       servo2.write(SERVO_Y_CENTRO);
94     }
95     else if (cmd == "UP_LEFT") {
96       servo2.write(SERVO_Y_MAX);
97       delay(500);
98       servo1.write(SERVO_X_MIN);
99     }
100    else if (cmd == "UP_RIGHT") {
101      servo2.write(SERVO_Y_MAX);
102      delay(500);
103      servo1.write(SERVO_X_MAX);
104    }
105    else if (cmd == "DOWN_LEFT") {
106      servo2.write(SERVO_Y_MIN);
107      delay(500);
108      servo1.write(SERVO_X_MIN);
109    }
110    else if (cmd == "DOWN_RIGHT") {
111      servo2.write(SERVO_Y_MIN);
112      delay(500);
113      servo1.write(SERVO_X_MAX);
114    }
115  }
116  delay(20);
117 }

```

Códigos del Android App Studio:

- Código 1:


```

package com.example.eyepilot_app

import android.bluetooth.BluetoothDevice
import android.bluetooth.BluetoothSocket
import android.util.Log
import java.io.OutputStream
import java.util.UUID

object BluetoothController {

    private const val TAG = "BT"
    private val uuid: UUID = UUID.fromString("00001101-0000-1000-8000-00805f9b34fb")

    private var socket: BluetoothSocket? = null
    private var outputStream: OutputStream? = null

    fun connect(device: BluetoothDevice) {
        Thread {
            try {
                Log.d(TAG, "Connecting to ${device.name}...")
                socket = device.createRfcommSocketToServiceRecord(uuid)
                socket?.connect()

                outputStream = socket?.outputStream
                Log.d(TAG, "✅ Bluetooth Connected!")
            } catch (e: Exception) {
                Log.e(TAG, "❌ Error connecting: ${e.message}")
            }
        }.start()
    }

    fun sendCommand(cmd: String) {
        Thread {
            try {
                outputStream?.write("$cmd\n".toByteArray())
                Log.d(TAG, "➡️ Command sent: $cmd")
            } catch (e: Exception) {
                Log.e(TAG, "❌ Error sending: ${e.message}")
            }
        }.start()
    }

    fun disconnect() {
        try {
            outputStream?.close()
            socket?.close()
            Log.d(TAG, "✅ Disconnected")
        } catch (e: Exception) {
            Log.e(TAG, "❌ Error disconnecting: ${e.message}")
        }
    }
}

```

- Código 2:

```
package com.example.eyepilot_app
```

```
import android.Manifest
```

```
import android.bluetooth.BluetoothAdapter
```

```
import android.bluetooth.BluetoothDevice
```

```
import android.bluetooth.BluetoothManager
```

```
import android.content.pm.PackageManager
```

```
import android.os.Build
```

```
import android.os.Bundle
```

```
import android.util.Log
```

```
import androidx.activity.ComponentActivity
```

```
import androidx.activity.compose.setContent
```

```
import androidx.activity.compose.rememberLauncherForActivityResult
```

```
import androidx.activity.result.contract.ActivityResultContracts
```

```
import androidx.camera.core.CameraSelector
```

```
import androidx.camera.core.Preview
```

```
import androidx.camera.lifecycle.ProcessCameraProvider
```

```
import androidx.camera.view.PreviewView
```

```
import androidx.compose.foundation.layout.*
```

```
import androidx.compose.material3.Scaffold
```

```
import androidx.compose.runtime.*
```

```
import androidx.compose.ui.Alignment
```

```
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.viewinterop.AndroidView
```

```
import androidx.core.content.ContextCompat
```

```
import androidx.lifecycle.compose.LocalLifecycleOwner
```

```
class MainActivity : ComponentActivity() {
```

```
    private var bluetoothAdapter: BluetoothAdapter? = null
```

```
    private val espMac = "00:22:04:AB:CD:EF"
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        setContent {
```

```
            CameraPreview(modifier = Modifier.fillMaxSize())
```

```
        }
```

```
        val bluetoothManager = getSystemService(BLUETOOTH_SERVICE) as BluetoothManager
```

```
        bluetoothAdapter = bluetoothManager.adapter
```

```
        requestBluetoothPermissions()
```

```
        connectToESP()
```

```
        setContent {
```

```
            Scaffold(modifier = Modifier.fillMaxSize()) { padding ->
```

```
val lifecycleOwner = LocalLifecycleOwner.current
```

```
var hasCameraPermission by remember { mutableStateOf(false) }
```

```
val cameraPermissionLauncher = rememberLauncherForActivityResult(  
    contract = ActivityResultContracts.RequestPermission(),  
    onResult = { granted: Boolean -> hasCameraPermission = granted }  
)
```

```
LaunchedEffect(Unit) {  
    cameraPermissionLauncher.launch(Manifest.permission.CAMERA)  
}
```

```
Box(  
    modifier = Modifier  
        .fillMaxSize()  
        .padding(padding)  
) {
```

```
    if (hasCameraPermission) {  
        CameraPreview(  
            modifier = Modifier.fillMaxSize(),  
            lifecycleOwner = lifecycleOwner  
        )  
    }
```

```
}
```

```
ControlsOverlay(
```

```
    modifier = Modifier.align(Alignment.Center),
```

```
    sendCommand = { cmd -> BluetoothController.sendCommand(cmd) }
```

```
)
```

```
}
```

```
}
```

```
}
```

```
}
```

```
private fun requestBluetoothPermissions() {
```

```
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
```

```
        val permissions = arrayOf(
```

```
            Manifest.permission.BLUETOOTH_CONNECT,
```

```
            Manifest.permission.BLUETOOTH_SCAN
```

```
)
```

```
    val missing = permissions.any {
```

```
        checkSelfPermission(it) != PackageManager.PERMISSION_GRANTED
```

```
}
```

```
    if (missing) {
```

```
        requestPermissions(permissions, 1001)
```

```
}
```

```

    }
}

private fun connectToESP() {
    try {
        val device: BluetoothDevice? = bluetoothAdapter?.getRemoteDevice(espMac)

        if (device != null) {
            BluetoothController.connect(device)
        } else {
            Log.e("BT", "Device not found")
        }
    } catch (e: SecurityException) {
        Log.e("BT", "Permission denied", e)
    }
}
}

```

@Composable

```

fun CameraPreview(
    modifier: Modifier,
    lifecycleOwner: androidx.lifecycle.LifecycleOwner
) {
    AndroidView(
        modifier = modifier,
        factory = { ctx ->

```

```

val previewView = PreviewView(ctx)

val cameraProviderFuture = ProcessCameraProvider.getInstance(ctx)

cameraProviderFuture.addListener({

    val cameraProvider = cameraProviderFuture.get()

    val preview = Preview.Builder().build().also {

        it.setSurfaceProvider(previewView.surfaceProvider)

    }

    try {

        cameraProvider.unbindAll()

        cameraProvider.bindToLifecycle(

            lifecycleOwner,

            CameraSelector.DEFAULT_BACK_CAMERA,

            preview

        )

    } catch (e: Exception) {

        e.printStackTrace()

    }

}, ContextCompat.getMainExecutor(ctx))

previewView

}

)

```

```
}
```

- Código 3:

```
package com.example.eyepilot_app
```

```
import androidx.compose.foundation.layout.*
```

```
import androidx.compose.foundation.shape.RoundedCornerShape
```

```
import androidx.compose.material3.Button
```

```
import androidx.compose.material3.ButtonDefaults
```

```
import androidx.compose.material3.Text
```

```
import androidx.compose.runtime.Composable
```

```
import androidx.compose.ui.Alignment
```

```
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.graphics.Color
```

```
import androidx.compose.ui.unit.dp
```

```
import androidx.compose.ui.unit.sp
```

```
import androidx.compose.ui.text.font.FontWeight
```

```
import androidx.compose.ui.res.painterResource
```

```
import androidx.compose.foundation.Image
```

```
@Composable
```

```
fun ControlsOverlay(modifier: Modifier = Modifier, sendCommand: (String) -> Unit) {
```



```
val buttonModifier = Modifier.size(80.dp).padding(5.dp)
```

```
val color = ButtonDefaults.buttonColors(containerColor = Color.Black.copy(alpha = 0.4f))
```

```
Column(
```

```
    modifier = modifier,
```

```
    horizontalAlignment = Alignment.CenterHorizontally,
```

```
    verticalArrangement = Arrangement.Center
```

```
) {
```

```
    Row {
```

```
        Button(onClick = { sendCommand("UP_LEFT") },
```

```
            modifier = buttonModifier, colors = color, shape = RoundedCornerShape(9.dp)) {
```

```
Text("↖", fontSize = 38.sp, fontWeight = FontWeight.Bold) }
```

```
        Button(onClick = { sendCommand("UP") },
```

```
            modifier = buttonModifier, colors = color, shape = RoundedCornerShape(9.dp)) {
```

```
Text("↑", fontSize = 38.sp, fontWeight = FontWeight.Bold) }
```

```
        Button(onClick = { sendCommand("UP_RIGHT") },
```

```
            modifier = buttonModifier, colors = color, shape = RoundedCornerShape(9.dp)) {
```

```
Text("↗", fontSize = 38.sp, fontWeight = FontWeight.Bold) }
```

```
    }
```

```
    Spacer(Modifier.height(12.dp))
```

```
    Row {
```

```
        Button(onClick = { sendCommand("LEFT") },
```

```
        modifier = buttonModifier, colors = color, shape = RoundedCornerShape(9.dp)) {  
Text("←", fontSize = 38.sp, fontWeight = FontWeight.Bold) }
```

```
    Button(onClick = { sendCommand("STOP") },
```

```
        modifier = buttonModifier, colors = color, shape = RoundedCornerShape(9.dp)) {  
Text("■", fontSize = 38.sp, fontWeight = FontWeight.Bold) }
```

```
    Button(onClick = { sendCommand("RIGHT") },
```

```
        modifier = buttonModifier, colors = color, shape = RoundedCornerShape(9.dp)) {  
Text("→", fontSize = 38.sp, fontWeight = FontWeight.Bold) }
```

```
}
```

```
Spacer(Modifier.height(12.dp))
```

```
Row {
```

```
    Button(onClick = { sendCommand("DOWN_LEFT") },
```

```
        modifier = buttonModifier, colors = color, shape = RoundedCornerShape(9.dp)) {  
Text("↙", fontSize = 38.sp, fontWeight = FontWeight.Bold) }
```

```
    Button(onClick = { sendCommand("DOWN") },
```

```
        modifier = buttonModifier, colors = color, shape = RoundedCornerShape(9.dp)) {  
Text("↓", fontSize = 38.sp, fontWeight = FontWeight.Bold) }
```

```
    Button(onClick = { sendCommand("DOWN_RIGHT") },
```

```
        modifier = buttonModifier, colors = color, shape = RoundedCornerShape(9.dp)) {  
Text("↘", fontSize = 38.sp, fontWeight = FontWeight.Bold) }
```

```
}
```

```
}
```

```
}
```